

# Security Audit Report

POS — Punto de Venta

|         |                                                                                                    |
|---------|----------------------------------------------------------------------------------------------------|
| Project | POS — Punto de Venta                                                                               |
| Date    | 2026-05-07                                                                                         |
| Version | V2026.05.07-005                                                                                    |
| Auditor | Claude Code (automated)                                                                            |
| Scope   | Auth & Access Control · Privacy Boundaries · API Security · Data Integrity<br>· Production Hygiene |

RISK SCORE

42/100

● RED

|          |      |        |     |      |
|----------|------|--------|-----|------|
| CRITICAL | HIGH | MEDIUM | LOW | INFO |
| 0        | 2    | 5      | 3   | 2    |

## Executive Summary

The POS system demonstrates solid foundational security practices: all endpoints are authenticated, input is validated via Pydantic models on every route, passwords are hashed with bcrypt, SQL injection is prevented via SQLAlchemy parameterized queries, and sensitive fields are excluded from API responses. The main risk areas are operational: rate limiting is completely absent (enabling brute-force attacks), audit logs cover sales and purchases but not user or settings mutations, and the default admin password is a known weak credential. For a single-location offline POS deployed on a trusted LAN, risk is moderate. For any internet-facing or multi-tenant deployment, the missing rate limiting and audit gaps must be addressed before go-live.

# Compliance Framework Scores

Scored for reference. HIPAA and CMMC are not in scope for this personal project. ISO 27001:2022 and SOC2 TSC scored as baseline security posture indicators.

| Framework      | Score   | Percentage | Status    |
|----------------|---------|------------|-----------|
| SOC2 TSC       | 7.5/28  | 26.8%      | X FAILING |
| HIPAA          | 6.5/18  | 36.1%      | X FAILING |
| CMMC L2        | 7.5/22  | 34.1%      | X FAILING |
| ISO 27001:2022 | 10.5/24 | 43.8%      | X FAILING |

## Scoring Details by Framework

### SOC2 TSC

| Control ID | Description                          | Score    |
|------------|--------------------------------------|----------|
| CC6.1      | Logical access controls              | Partial  |
| CC6.2      | Authentication mechanisms            | Partial  |
| CC6.3      | Role-based access                    | Complete |
| CC6.4      | Auth activity logging                | Partial  |
| CC6.5      | Account termination / session expiry | Partial  |
| CC6.6      | Network protection                   | Partial  |
| CC6.7      | Encryption in transit                | Partial  |
| CC6.8      | Input validation                     | Complete |
| CC7.1      | Threat / anomaly detection           | Absent   |
| CC7.2      | System monitoring                    | Partial  |
| CC7.3      | Security evaluation                  | Absent   |
| CC7.4      | Incident response                    | Absent   |
| CC7.5      | Recovery                             | Absent   |
| CC8.1      | Change management                    | Partial  |
| CC8.2      | SDLC security gates                  | Absent   |
| CC9.1      | Risk assessment                      | Absent   |
| CC9.2      | Vendor / third-party risk            | Partial  |
| A1.1       | Availability monitoring              | Partial  |
| A1.2       | Environmental protection             | Absent   |
| A1.3       | Recovery testing                     | Absent   |

### HIPAA

| Control ID | Description | Score |
|------------|-------------|-------|
|------------|-------------|-------|

|                      |                                     |          |
|----------------------|-------------------------------------|----------|
| 164.308(a)(1)(i)     | Security management — risk analysis | Absent   |
| 164.308(a)(1)(ii)(D) | Information system activity review  | Partial  |
| 164.308(a)(3)(ii)(A) | Authorization / supervision         | Complete |
| 164.308(a)(4)(ii)(B) | Access authorization                | Complete |
| 164.308(a)(5)(ii)(C) | Log-in monitoring / MFA             | Absent   |
| 164.308(a)(5)(ii)(D) | Password management                 | Partial  |
| 164.310(a)(2)(ii)    | Workstation use policy              | Absent   |
| 164.310(d)(1)        | Device and media controls           | Absent   |
| 164.312(a)(1)        | Unique user identification          | Complete |
| 164.312(a)(2)(i)     | Emergency access procedure          | Absent   |
| 164.312(a)(2)(iii)   | Automatic logoff                    | Partial  |
| 164.312(b)           | Audit controls                      | Partial  |
| 164.312(c)(1)        | Integrity — authentication          | Partial  |
| 164.312(d)           | Person / entity authentication      | Partial  |
| 164.312(e)(1)        | Transmission security               | Partial  |
| 164.314(a)(2)(i)     | Business associate contracts        | Absent   |
| 164.314(b)(2)(i)     | Group health plan separation        | Absent   |
| 164.316(b)(1)        | Documentation policies              | Absent   |

## CMMC L2

| Control ID | Description                                            | Score    |
|------------|--------------------------------------------------------|----------|
| AC.1.001   | Limit system access to authorized users                | Complete |
| AC.1.002   | Limit system access to authorized functions            | Partial  |
| AC.2.006   | Control access to CUI in non-federal systems           | Partial  |
| AC.2.007   | Employ principle of least privilege                    | Partial  |
| AC.2.147   | Authorize remote execution                             | Partial  |
| AU.2.041   | Ensure individual actions are traceable                | Partial  |
| AU.2.042   | Create and retain system audit logs                    | Partial  |
| CM.3.165   | Configure systems to detect/prevent malicious activity | Absent   |
| IA.1.077   | Authenticate users before allowing system access       | Partial  |
| IA.3.083   | Use multifactor authentication                         | Absent   |
| IR.2.092   | Establish incident-handling capability                 | Absent   |
| RA.3.137   | Periodically assess risk                               | Absent   |
| SC.1.175   | Monitor communications at boundaries                   | Partial  |
| SC.3.177   | Employ FIPS-validated cryptography                     | Absent   |
| SC.3.178   | Control connections to external systems                | Partial  |
| SC.3.181   | Protect communications of CUI                          | Partial  |
| SC.3.187   | Establish and manage cryptographic keys                | Partial  |
| SI.1.210   | Identify, report, and correct system flaws             | Partial  |
| SI.3.171   | Implement spam and malicious code protection           | Absent   |

|          |                                           |         |
|----------|-------------------------------------------|---------|
| SI.4.173 | Real-time alert on attack indicators      | Absent  |
| SI.4.177 | Identify unauthorized use of systems      | Partial |
| SI.4.178 | Analyze system behavior to detect attacks | Absent  |

## ISO 27001:2022

| Control ID | Description                                    | Score    |
|------------|------------------------------------------------|----------|
| A.5.15     | Access control policy                          | Partial  |
| A.5.16     | Identity management                            | Complete |
| A.5.17     | Authentication information                     | Partial  |
| A.5.18     | Access rights                                  | Complete |
| A.5.19     | Information security in supplier relationships | Absent   |
| A.5.31     | Legal, statutory, regulatory requirements      | Absent   |
| A.8.2      | Privileged access rights                       | Partial  |
| A.8.3      | Information access restriction                 | Partial  |
| A.8.4      | Access to source code                          | Partial  |
| A.8.5      | Secure authentication                          | Partial  |
| A.8.7      | Protection against malware                     | Absent   |
| A.8.9      | Configuration management                       | Partial  |
| A.8.12     | Data leakage prevention                        | Partial  |
| A.8.15     | Logging                                        | Partial  |
| A.8.16     | Monitoring activities                          | Absent   |
| A.8.20     | Network security                               | Partial  |
| A.8.21     | Security of network services                   | Partial  |
| A.8.22     | Segregation of networks                        | Partial  |
| A.8.24     | Use of cryptography                            | Partial  |
| A.8.25     | Secure development lifecycle                   | Absent   |
| A.8.26     | Application security requirements              | Partial  |
| A.8.27     | Secure system architecture                     | Partial  |
| A.8.28     | Secure coding                                  | Complete |
| A.8.29     | Security testing in development                | Absent   |

# Security Findings

## ■ HIGH

### [F001] Rate limiting completely absent

**What:** No rate limiting middleware exists on any endpoint — auth, read, or write

**Location:** backend/app/main.py (all routers)

**Risk:** An attacker can brute-force the /api/v1/auth/login endpoint indefinitely, enumerate user accounts, or launch a denial-of-service attack against any endpoint without throttling

**Fix:** Add slowapi middleware. Apply 5 req/min to /api/v1/auth/login, 60 req/min to write endpoints, 300 req/min to read endpoints. Store rate limit state in memory or Redis.

**SOC2:** CC6.1, CC6.6 **HIPAA:** 164.312(a)(2)(i) **CMMC:** AC.2.006, SC.3.178 **ISO27001:** A.12.6.1, A.13.1.1

### [F002] Audit logs absent for user mutations and settings changes

**What:** No audit log entries created when users are created/modified/deleted or when business settings change

**Location:** backend/app/routers/users.py, backend/app/routers/settings.py

**Risk:** A compromised admin account could modify users or settings with no forensic trail. Cannot detect unauthorized privilege escalation or configuration tampering after the fact

**Fix:** Add audit\_log entries in user\_service (create, update, delete) and settings\_service (update). Use existing AuditLog model. Log: actor\_id, action, entity\_type, entity\_id, payload diff, timestamp.

**SOC2:** CC6.4, CC7.2 **HIPAA:** 164.312(b), 164.308(a)(1)(ii)(D) **CMMC:** AU.2.041, AU.2.042 **ISO27001:** A.12.4.1, A.12.4.3

## ■ MEDIUM

### [F003] Default admin password 'Admin123!' hardcoded as fallback

**What:** ADMIN\_INITIAL\_PASSWORD defaults to 'Admin123!' in config.py and is present in the .env file

**Location:** backend/app/config.py:49, .env:57

**Risk:** If the install script is skipped or .env not customized, the admin account has a known default password attackable via credential stuffing

**Fix:** Remove default value from config.py (make it required). Force password change on first login. Generate a random password in the install script and display it once only.

**SOC2:** CC6.2 **HIPAA:** 164.308(a)(5)(ii)(D) **CMMC:** IA.1.077 **ISO27001:** A.9.4.3

#### [F004] JWT access token expiry set to 8 hours (480 minutes)

**What:** ACCESS\_TOKEN\_EXPIRE\_MINUTES defaults to 480 — tokens remain valid for a full work shift even after logout

**Location:** backend/app/config.py:14

**Risk:** A stolen token (via XSS, network sniff, or log leak) remains valid for up to 8 hours. No token revocation mechanism exists to invalidate a token before expiry.

**Fix:** Reduce to 60 minutes. Implement refresh-token rotation or a token revocation list (Redis set of invalidated JTIs). Log out should invalidate the token server-side.

**SOC2:** CC6.5 **HIPAA:** 164.312(a)(2)(iii) **CMMC:** AC.1.002, IA.3.083 **ISO27001:** A.9.4.2

#### [F005] SSRF risk: logo\_url downloaded server-side without domain allowlist

**What:** Print Bridge fetches logo\_url via urllib.request.urlopen with no domain validation, and backend passes any logo\_url stored in business settings

**Location:** print\_bridge/main.py (\_gdi\_draw\_logo), backend/app/routers/sales.py:282-287

**Risk:** An admin-level attacker can set logo\_url to an internal network address (e.g., http://192.168.1.1/admin) causing Print Bridge to probe internal services. Also enables request forgery against internal HTTP endpoints.

**Fix:** Validate logo\_url is HTTPS with an allowlisted domain in business\_settings validator. In Print Bridge, also validate URL scheme and hostname before fetching. Add 5-second timeout and 2MB max size (already has 5s timeout — add size limit).

**SOC2:** CC6.2, CC6.6 **HIPAA:** — **CMMC:** SC.3.178, SI.3.171 **ISO27001:** A.13.1.1, A.14.2.1

#### [F006] Missing CSP and HSTS headers in Caddy reverse proxy

**What:** Caddyfile includes X-Frame-Options, X-Content-Type-Options, Referrer-Policy, X-XSS-Protection but is missing Content-Security-Policy and Strict-Transport-Security

**Location:** caddy/Caddyfile:17-25

**Risk:** Absence of CSP leaves XSS attacks unmitigated at transport layer. Missing HSTS means if HTTPS is ever enabled, browsers will not enforce it, allowing SSL-strip attacks.

**Fix:** Add: header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; font-src 'self'". Add HSTS when HTTPS is configured.

**SOC2:** CC6.1, CC6.6 **HIPAA:** 164.312(a)(2)(i) **CMMC:** SC.3.181 **ISO27001:** A.14.2.1

#### [F007] Tenant isolation not enforced at DB query level

**What:** All service queries do not filter by tenant\_id. Models have tenant\_id with a hardcoded default UUID but queries never enforce tenant boundaries

**Location:** backend/app/services/\*.py (multiple service files)

**Risk:** If multi-tenant deployment is ever attempted, all data from all tenants would be accessible to any authenticated user. Also allows future IDOR bugs if tenant\_id filtering is added inconsistently.

**Fix:** Add tenant\_id to all WHERE clauses now, even as single-tenant. Create a TenantContext dependency that injects tenant\_id from the authenticated user. Fail closed: 404 if entity.tenant\_id != current\_user.tenant\_id.

**SOC2:** CC6.1, CC6.3 **HIPAA:** 164.312(a)(1) **CMMC:** AC.2.147 **ISO27001:** A.9.1.2

## ■ LOW

### [F008] SQLAlchemy echo enabled in non-production environments

**What:** database.py uses echo=not settings.is\_production, logging all SQL statements including WHERE clause values

**Location:** backend/app/database.py:8

**Risk:** SQL logs can expose customer names, product names, and partial financial data. If development logs are forwarded to a log aggregator, PII leaks to that system.

**Fix:** Set echo=False by default. Add a separate DEBUG\_SQL=false env var to enable selectively. Never enable echo in environments that process real customer data.

**SOC2:** CC7.2 **HIPAA:** 164.312(b) **CMMC:** AU.2.041 **ISO27001:** A.12.4.1

### [F009] backend/.env.test committed to version control

**What:** backend/.env.test contains test database credentials and a fixed SECRET\_KEY and is tracked by git

**Location:** backend/.env.test

**Risk:** Although credentials are for a test DB (not real secrets), this establishes a bad pattern. It also exposes the exact SECRET\_KEY format expected. Developers may accidentally base production secrets on test values.

**Fix:** Add backend/.env.test to .gitignore. Use environment variables or a CI secret for test runs. Provide a .env.test.example with placeholder values instead.

**SOC2:** CC6.2 **HIPAA:** — **CMMC:** CM.3.165 **ISO27001:** A.14.2.3

### [F010] next.config.ts has no security headers configuration

**What:** Frontend Next.js app defines no headers() function in next.config.ts — relies entirely on the Caddy proxy for security headers

**Location:** frontend/next.config.ts

**Risk:** If frontend is accessed directly (dev server, behind a different proxy, or Caddy misconfiguration), browsers receive no security headers. Especially relevant during development where Caddy is not used.

**Fix:** Add headers() export to next.config.ts with X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and CSP. Defense-in-depth: headers at both proxy and app level.

**SOC2:** CC6.1 **HIPAA:** — **CMMC:** SC.3.181 **ISO27001:** A.14.2.1

## ■ ■ INFO

### [F011] CORS allows all methods and headers (origins correctly restricted)

**What:** CORSMiddleware uses allow\_methods=['\*'] and allow\_headers=['\*']. Origins are properly restricted to configured env var (default: localhost only)

**Location:** backend/app/main.py:44-45

**Risk:** Low risk given origin restriction. However, wildcard methods/headers in combination with allow\_credentials=True could be tightened. If origins are misconfigured to '\*' in production, this becomes exploitable.

**Fix:** Consider restricting to explicit methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'] and headers: ['Content-Type', 'Authorization']. Verify CORS\_ORIGINS env var is always set explicitly in production deployments.

**SOC2:** CC6.6 **HIPAA:** — **CMMC:** SC.3.178 **ISO27001:** A.13.1.1

### [F012] No multi-factor authentication for admin or supervisor accounts

**What:** Authentication is username+password only. No TOTP, hardware key, or second factor exists for privileged roles.

**Location:** backend/app/routers/auth.py

**Risk:** If admin credentials are compromised (phishing, reuse), an attacker gains full system access with no second barrier. Especially relevant since admin can void sales, modify settings, and view all reports.

**Fix:** Add TOTP (Google Authenticator/Authy) as an optional second factor, required for admin and supervisor roles. Libraries: pyotp + QR code enrollment flow.

**SOC2:** CC6.2 **HIPAA:** 164.308(a)(5)(ii)(C) **CMMC:** IA.3.083 **ISO27001:** A.9.4.2

# Remediation Priority

Address findings in this order:

| #  | Severity | Finding | Action                                                                     | Effort   |
|----|----------|---------|----------------------------------------------------------------------------|----------|
| 1  | HIGH     | F001    | Add slowapi rate limiting to auth + write endpoints                        | ~2 hours |
| 2  | HIGH     | F002    | Add audit log entries to user_service and settings_service                 | ~3 hours |
| 3  | MEDIUM   | F003    | Remove default admin password from config.py; force first-login change     | ~1 hour  |
| 4  | MEDIUM   | F006    | Add CSP header to Caddyfile                                                | ~30 min  |
| 5  | MEDIUM   | F004    | Reduce JWT expiry to 60 min; add refresh token flow                        | ~4 hours |
| 6  | MEDIUM   | F005    | Validate logo_url domain in business settings + size limit in Print Bridge | ~1 hour  |
| 7  | MEDIUM   | F007    | Add tenant_id filter to all service queries                                | ~2 hours |
| 8  | LOW      | F009    | Add backend/.env.test to .gitignore                                        | ~10 min  |
| 9  | LOW      | F010    | Add security headers to next.config.ts                                     | ~30 min  |
| 10 | LOW      | F008    | Set SQLAlchemy echo=False; add DEBUG_SQL env var                           | ~15 min  |

## Confirmed Security Controls ■

- bcrypt password hashing (passlib) — no plain-text or MD5 passwords
- Pydantic validation on ALL API routes — no unvalidated user input reaches the DB
- SQLAlchemy parameterized queries throughout — zero SQL injection surface
- Sensitive fields excluded from API responses (password hashes, internal IDs)
- 3-tier RBAC enforced on every route: cashier / supervisor / admin
- JWT authentication required on all endpoints except /health and /api/v1/branding
- FastAPI docs\_url / redoc\_url disabled in production (ENV=production)
- Docker images use pinned versions (not :latest)
- Non-root users in all Docker containers (appuser, nextjs)
- Structured logging (structlog) — no credentials or PII in logs
- Docker Compose isolates services on internal network — DB not exposed externally
- .env excluded from git (.gitignore verified)
- Audit logs present for all sales, purchases, returns, and gift card operations
- Soft deletes (deleted\_at) consistently used — no hard data deletion
- UUID primary keys — no sequential ID enumeration possible